Homework 3 Solution

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat

np.random.seed(0)
HW = ''   # location of the original data
plt.rcParams.update({'font.size': 11})
```

Problem 1: PCA on the 100 by 2000 data set

Setup

The data matrix $X \in \mathbb{R}^{100 \times 2000}$ has 100 samples (rows) and 2000 features (columns). PCA centers each feature, $X_c = X - \mathbf{1}\bar{x}^\top$, and takes the SVD $X_c = U\Sigma V^\top$. The rank-r approximation is

$$\tilde{X}_r = U_r \Sigma_r V_r^\top + \mathbf{1}\bar{x}^\top.$$

Because there are only 100 samples, $\operatorname{rank}(X_c) \le 99$. Therefore, for every $r \ge 99$ the reconstruction is exact. The prescribed list `r = np.arange(1,1000,5)` goes beyond this, so we cap the number of components at $\min(r, 100)$.

```
X = np.load(HW + 'hwk3data.npy')
print('shape:', X.shape)
mu = X.mean(axis=0)
Xc = X - mu
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
print('number of singular values:', len(s), ' numerical rank:', np.sum(s > 1e-10*s[0]))

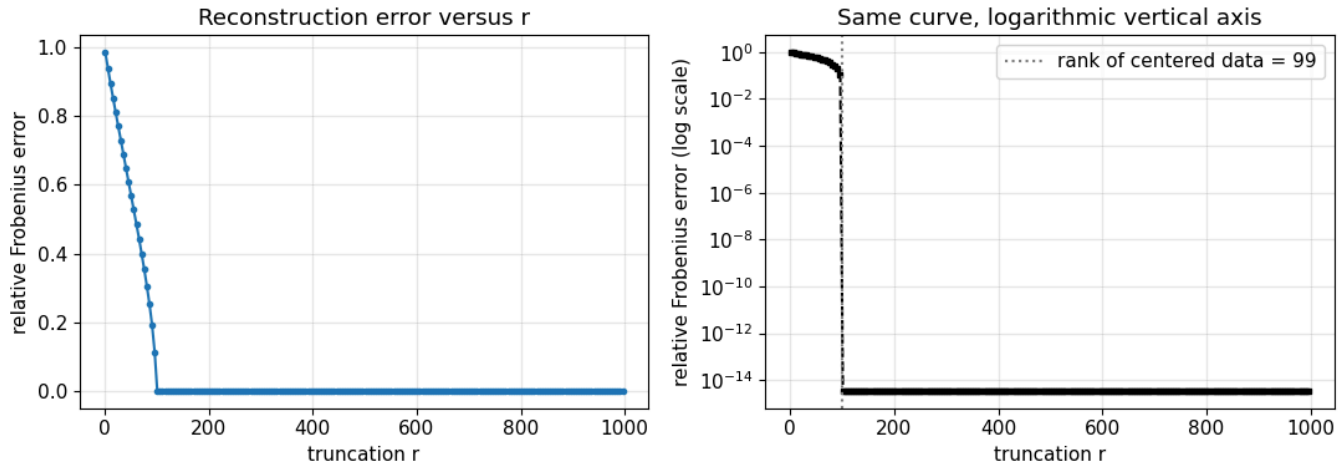
rs = np.arange(1, 1000, 5)
err_fro = []
for r in rs:
    k = min(r, len(s))
    Xr = (U[:, :k] * s[:k]) @ Vt[:k] + mu
    err_fro.append(np.linalg.norm(X - Xr, 'fro') / np.linalg.norm(X, 'fro'))
err_fro = np.array(err_fro)
# cross-check with singular values: ||Xc - Xc_r||_F^2 = sum of discarded sigma^2
tail = np.array([np.sum(s[min(r,len(s)):]**2) for r in rs])
print('max |direct - from sigma| (squared absolute error):',
      np.max(np.abs((err_fro*np.linalg.norm(X,'fro'))**2 - tail)))
for r in [1, 6, 11, 51, 96, 101]:
    print(f'r={r:4d}  relative error (vs ||X||_F) = {err_fro[list(rs).index(r)]:.4e}')
```

```
shape: (100, 2000)
```

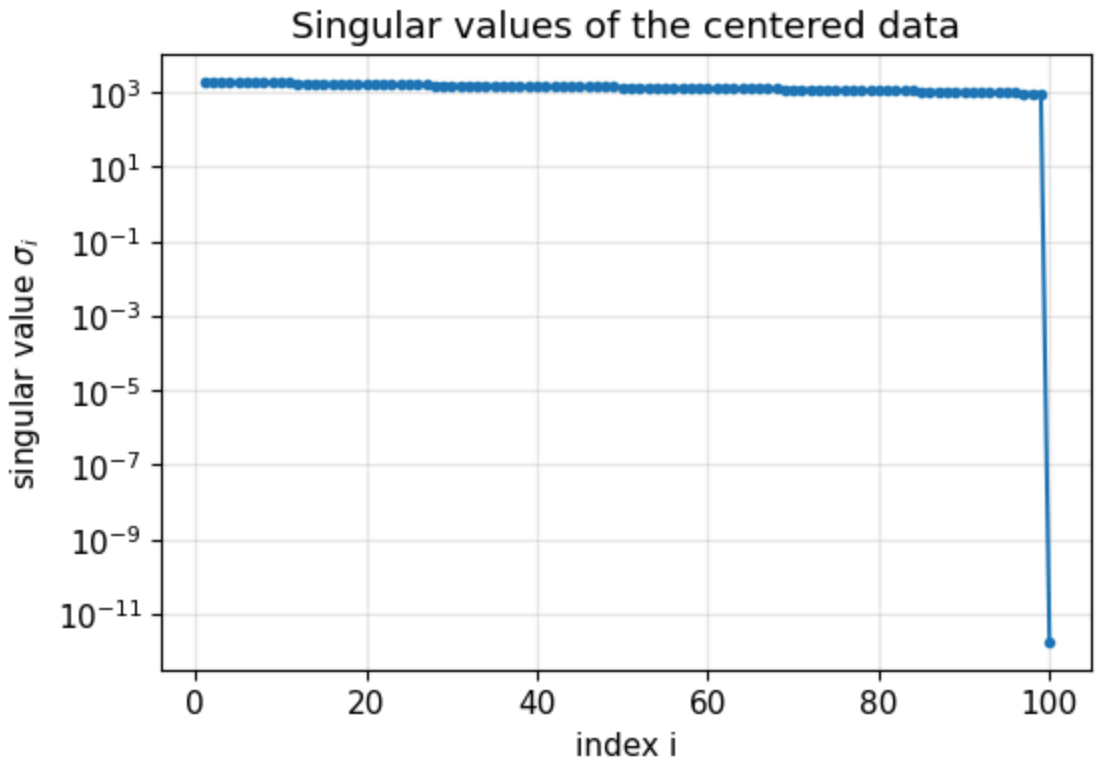
```
number of singular values: 100  numerical rank: 99
```

```
max |direct − from sigma| (squared absolute error): 1.1026859283447266e-06
r=   1  relative error (vs ||X||_F) = 9.8597e-01
r=   6  relative error (vs ||X||_F) = 9.3935e-01
r=  11  relative error (vs ||X||_F) = 8.9524e-01
r=  51  relative error (vs ||X||_F) = 5.6872e-01
r=  96  relative error (vs ||X||_F) = 1.1427e-01
r= 101  relative error (vs ||X||_F) = 3.3854e-15
```

```
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(rs, err_fro, 'o-', ms=3)
ax[0].set_xlabel('truncation r'); ax[0].set_ylabel('relative Frobenius error')
ax[0].set_title('Reconstruction error versus r'); ax[0].grid(alpha=.3)
ax[1].semilogy(rs, err_fro + 1e-17, 's--', ms=3, color='k')
ax[1].axvline(99, ls=':', color='gray', label='rank of centered data = 99')
ax[1].set_xlabel('truncation r'); ax[1].set_ylabel('relative Frobenius error (log scale)')
ax[1].set_title('Same curve, logarithmic vertical axis'); ax[1].legend();
ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
```




```
plt.figure(figsize=(6,4))
plt.semilogy(np.arange(1,len(s)+1), s, 'o-', ms=3)
plt.xlabel('index i'); plt.ylabel(r'singular value $\sigma_i$')
plt.title('Singular values of the centered data'); plt.grid(alpha=.3); plt.show()
```



Problem 2: SVD of flow past a cylinder

Load and inspect the data

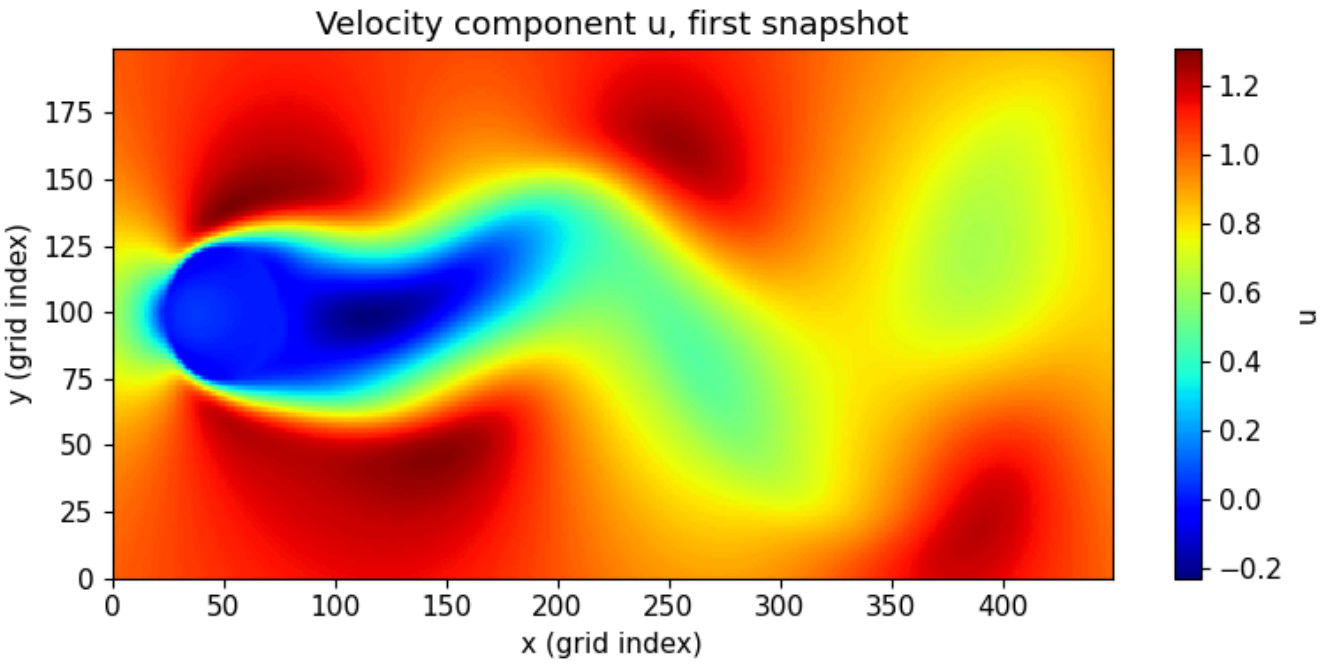
The file contains UALL, VALL (velocity components, 89351 by 151) and VORTALL (vorticity), each column being one snapshot of a 449×199 grid, plus a single extra snapshot for each (UEXTRA, VEXTRA, VORTEXTRA) that is not part of the 151 columns.

We take the data matrix to be $X =$ UALL, i.e. the u-velocity movie with $m = 89351$ grid points and 151 snapshots.

```
d = loadmat(HW + 'CYLINDER_ALL.mat')
X = d['UALL'];  nx, ny = d['nx'].item(), d['ny'].item()
print('X shape:', X.shape, ' grid:', nx, 'x', ny, ' nx*ny =', nx*ny)
u = X.reshape(449, 199, -1)
uextra = d['UEXTRA'].reshape(449, 199, -1)   # corrected name (snippet swaps U/V)
print('u cube:', u.shape, ' extra snapshot:', uextra.shape)

def show(ax, field, title, vmin=None, vmax=None):
    im = ax.pcolormesh(field.T, cmap='jet', vmin=vmin, vmax=vmax)
    ax.set_title(title); ax.set_xlabel('x (grid index)'); ax.set_ylabel('y (grid index)')
    return im
fig, ax = plt.subplots(figsize=(8,4))
im = show(ax, u[:,:,0], 'Velocity component u, first snapshot'); plt.colorbar(im, ax=ax,
label='u')
plt.tight_layout(); plt.show()
```

```
X shape: (89351, 151)  grid: 199 x 449  nx*ny = 89351
u cube: (449, 199, 151)  extra snapshot: (449, 199, 1)
```

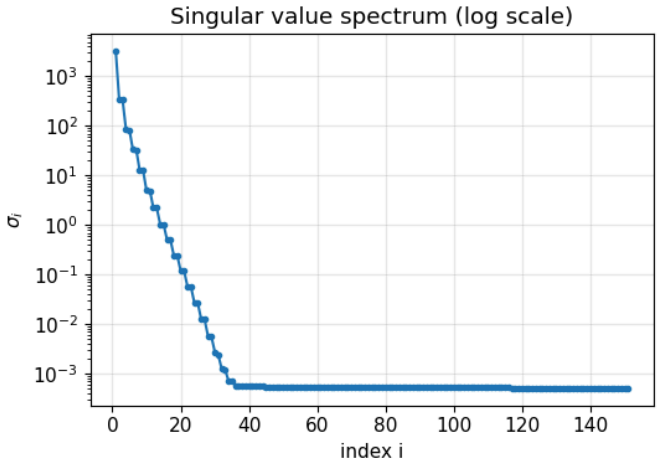


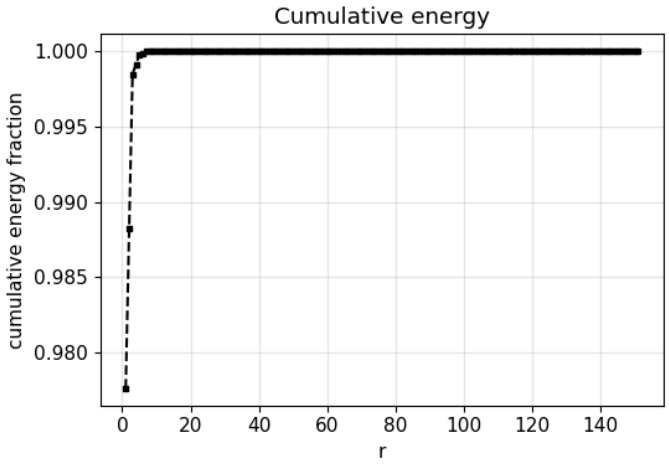
(a) SVD, singular value spectrum and leading modes

We compute the economy SVD $X = U\Sigma V^*$ without subtracting the mean, so the first mode captures the mean flow.

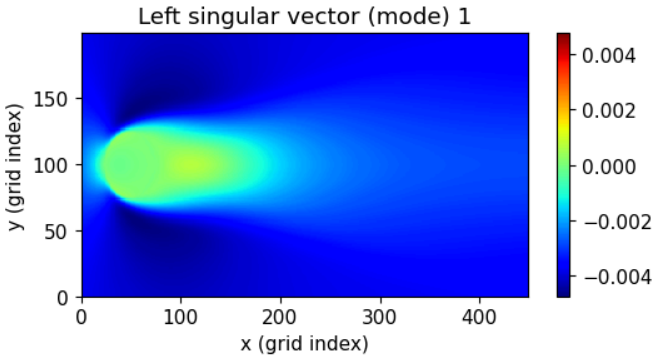
```
U, s, Vt = np.linalg.svd(X, full_matrices=False)
print(U.shape, s.shape, Vt.shape)
print('first 6 singular values:', np.round(s[:6], 2))
fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].semilogy(np.arange(1,len(s)+1), s, 'o-', ms=3)
ax[0].set_xlabel('index i'); ax[0].set_ylabel(r'$\sigma_i$'); ax[0].set_title('Singular
value spectrum (log scale)'); ax[0].grid(alpha=.3)
ax[1].plot(np.arange(1,len(s)+1), np.cumsum(s**2)/np.sum(s**2), 's--', ms=3, color='k')
ax[1].set_xlabel('r'); ax[1].set_ylabel('cumulative energy fraction');
ax[1].set_title('Cumulative energy'); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
```

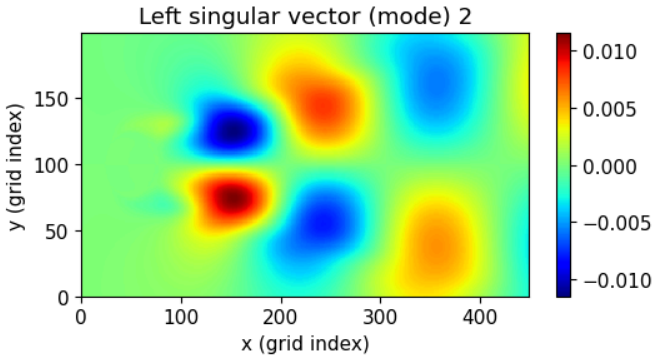
```
(89351, 151) (151,) (151, 151)
first 6 singular values: [3265.48  339.71  334.69   84.57   83.6    33.48]
```

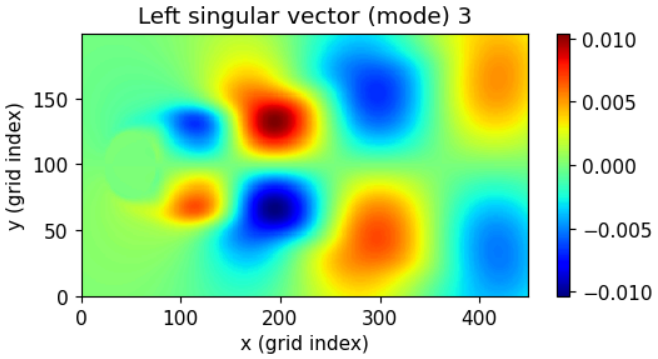


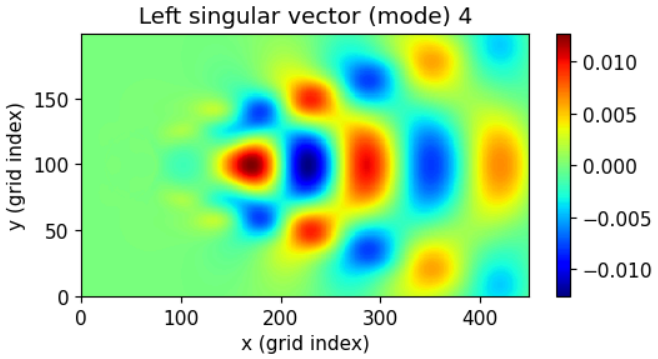


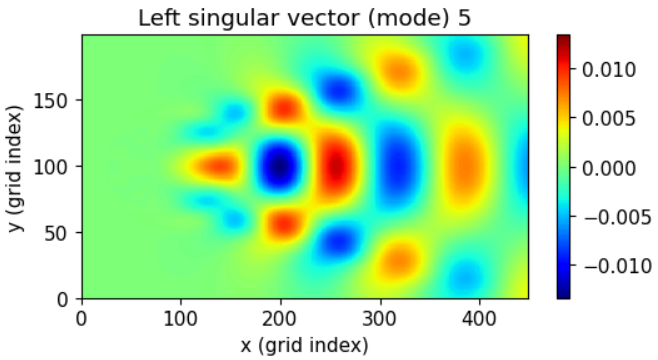

```
fig, axs = plt.subplots(3, 2, figsize=(11, 9))
for i, ax in enumerate(axs.ravel()):
    mode = U[:, i].reshape(449, 199)
    v = np.max(np.abs(mode))
    im = show(ax, mode, f'Left singular vector (mode) {i+1}', -v, v)
    plt.colorbar(im, ax=ax)
plt.tight_layout(); plt.show()
```

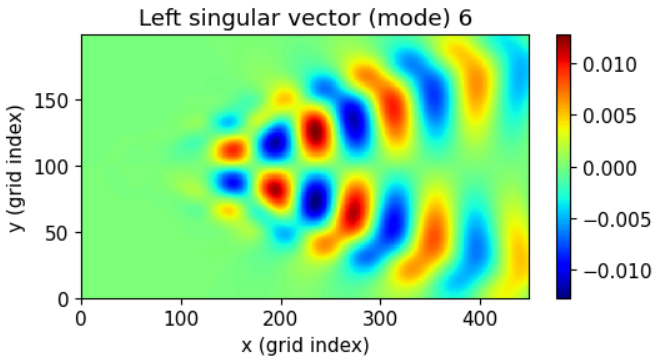












(b) Reconstructions, energy thresholds and error

The rank-r reconstruction is $\tilde{X}_r = U_r \Sigma_r V_r^*$. By the Eckart-Young theorem the squared Frobenius error is $\| X - \tilde{X}_r \|_F^2 = \sum_{i>r} \sigma_i^2$, and the energy is $\| X \|_F^2 = \sum_i \sigma_i^2$. The smallest r capturing a fraction p of the energy satisfies $\sum_{i \le r} \sigma_i^2 \ge p \, \| X \|_F^2$.

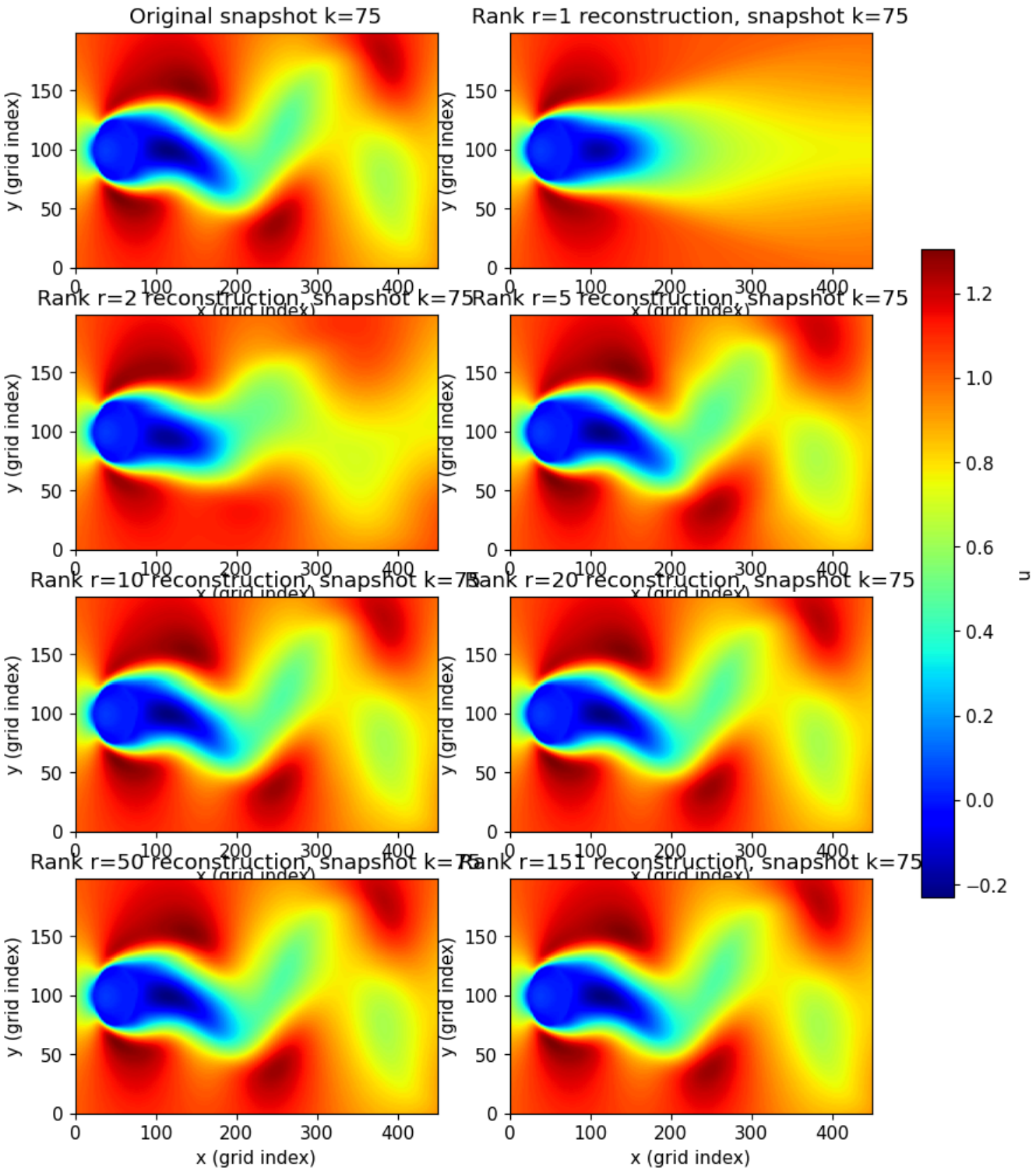
```
energy = np.cumsum(s**2) / np.sum(s**2)
for p in [0.90, 0.99, 0.999]:
    r_p = int(np.searchsorted(energy, p) + 1)
    Xr = (U[:, :r_p] * s[:r_p]) @ Vt[:r_p]
    err_direct = np.linalg.norm(X - Xr, 'fro')**2
    err_sigma = np.sum(s[r_p:]**2)
    print(f'{p*100:5.1f}% energy: r = {r_p:3d},  ||X - Xr||_F^2 = {err_direct:.4e} (from
sigma: {err_sigma:.4e}),'
          f' relative = {err_direct/np.linalg.norm(X,"fro")**2:.2e}')
print('total energy ||X||_F^2 =', f'{np.linalg.norm(X,"fro")**2:.4e}')
```

```
 90.0% energy: r =   1,  ||X - Xr||_F^2 = 2.4417e+05 (from sigma: 2.4417e+05), relative =
2.24e-02
```

```
 99.0% energy: r =   3,  ||X - Xr||_F^2 = 1.6753e+04 (from sigma: 1.6753e+04), relative =
1.54e-03
```

```
 99.9% energy: r =   4,  ||X - Xr||_F^2 = 9.6012e+03 (from sigma: 9.6012e+03), relative =
8.80e-04
total energy ||X||_F^2 = 1.0908e+07
```

```
rs_show = [1, 2, 5, 10, 20, 50]
k = 75   # snapshot (frame) shown
fig, axs = plt.subplots(4, 2, figsize=(11, 12))
axs = axs.ravel()
vmin, vmax = X[:, k].min(), X[:, k].max()
im = show(axs[0], X[:, k].reshape(449,199), f'Original snapshot k={k}', vmin, vmax)
for ax, r in zip(axs[1:], rs_show + [151]):
    Xr_k = (U[:, :r] * s[:r]) @ Vt[:r, k]
    show(ax, Xr_k.reshape(449,199), f'Rank r={r} reconstruction, snapshot k={k}', vmin,
vmax)
fig.colorbar(im, ax=axs, label='u', shrink=.6)
plt.show()
```




```
# "Movie" as a function: reconstruct all frames for a given r (frames could be passed to
matplotlib.animation).
def reconstruct_movie(r):
    return ((U[:, :r] * s[:r]) @ Vt[:r]).reshape(449, 199, -1)
for r in [1, 5, 10, 20, 50, 100, 151]:
    err = np.sum(s[r:]**2)
    print(f'r={r:3d}  squared Frobenius error = {err:.4e}   relative =
{err/np.sum(s**2):.3e}')
print('check movie shape:', reconstruct_movie(10).shape)
```

```
r=  1  squared Frobenius error = 2.4417e+05   relative = 2.239e-02
r=  5  squared Frobenius error = 2.6121e+03   relative = 2.395e-04
r= 10  squared Frobenius error = 3.7405e+01   relative = 3.429e-06
r= 20  squared Frobenius error = 2.2740e-02   relative = 2.085e-09
r= 50  squared Frobenius error = 2.7915e-05   relative = 2.559e-12
r=100  squared Frobenius error = 1.3565e-05   relative = 1.244e-12
r=151  squared Frobenius error = 0.0000e+00   relative = 0.000e+00
check movie shape: (449, 199, 151)
```

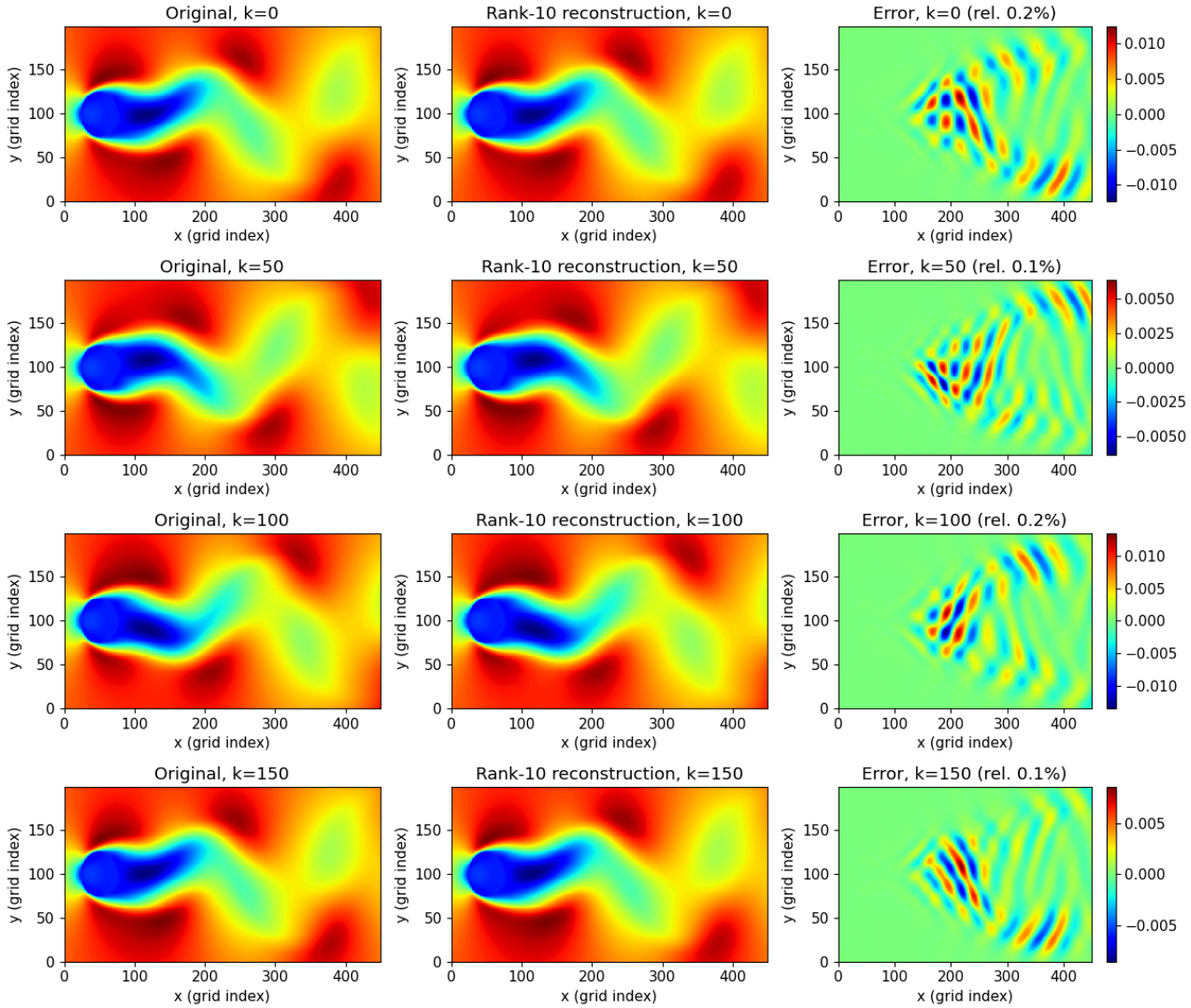
(c) Rank-10 truncation and the coefficient matrix W

With $r = 10$ let $\tilde{U} = U_{:,1:10}$, $\tilde{\Sigma} = \mathrm{diag}(\sigma_1, ..., \sigma_{10})$ and $\tilde{V} = V_{:,1:10}$. The matrix $W = \tilde{\Sigma}\tilde{V}^* \in \mathbb{R}^{10\times 151}$ has columns w_k giving the mixture of the 10 eigenflows (columns of $\tilde{U}$) in snapshot k, so $x_k \approx \tilde{U} w_k$.

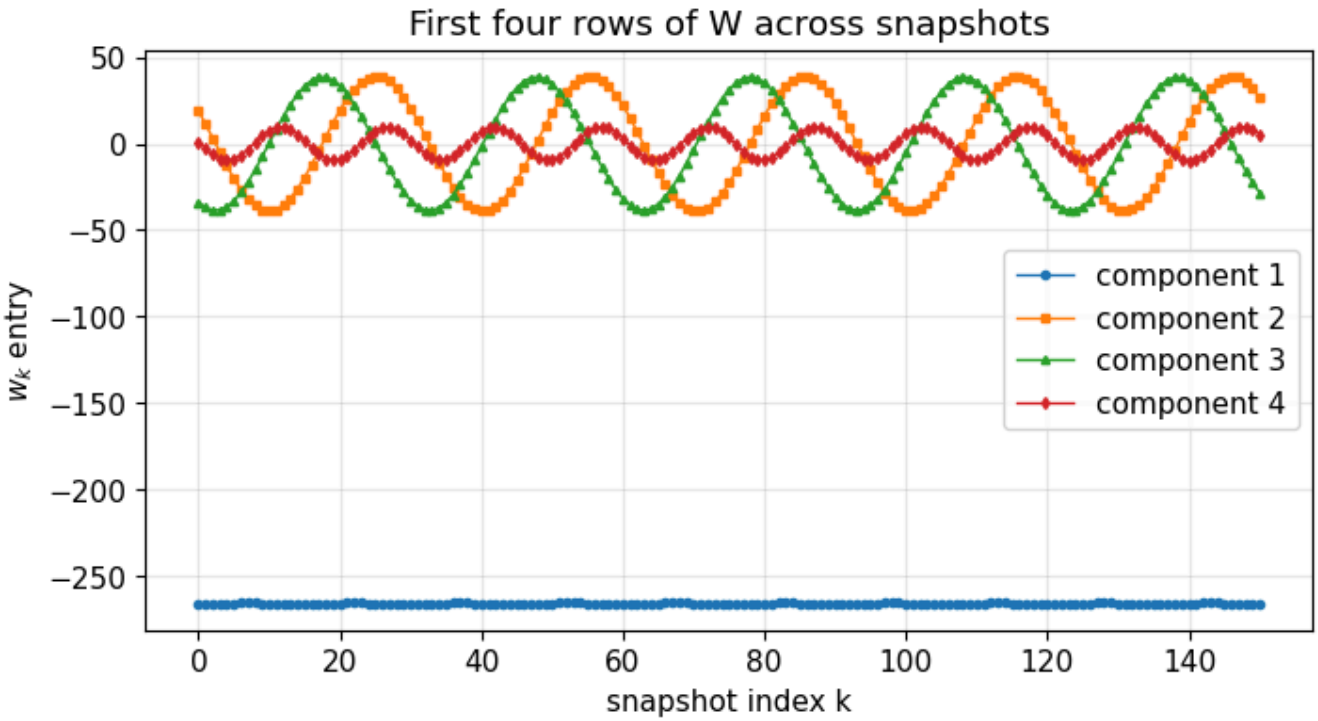
```
r = 10
Ut, St, Vtt = U[:, :r], np.diag(s[:r]), Vt[:r]
W = St @ Vtt
print('W shape:', W.shape)
Xtilde = Ut @ W
print('||X - Xtilde||_F^2 =', f'{np.linalg.norm(X-Xtilde,"fro")**2:.4e}', ' sum of
discarded sigma^2 =', f'{np.sum(s[r:]**2):.4e}')
ks = [0, 50, 100, 150]
fig, axs = plt.subplots(len(ks), 3, figsize=(13, 11))
for row, k in zip(axs, ks):
    a, b = X[:, k], Xtilde[:, k]
    vmin, vmax = a.min(), a.max()
    show(row[0], a.reshape(449,199), f'Original, k={k}', vmin, vmax)
    show(row[1], b.reshape(449,199), f'Rank-10 reconstruction, k={k}', vmin, vmax)
    e = (a-b).reshape(449,199); m_ = np.abs(e).max()
    im = show(row[2], e, f'Error, k={k} (rel. {np.linalg.norm(a-
b)/np.linalg.norm(a):.1%})', -m_, m_)
    plt.colorbar(im, ax=row[2])
plt.tight_layout(); plt.show()
```

```
W shape: (10, 151)
```

```
||X - Xtilde||_F^2 = 3.7405e+01  sum of discarded sigma^2 = 3.7405e+01
```




```
fig, ax = plt.subplots(figsize=(8,4))
for i, mk in zip(range(4), ['o','s','^','d']):
    ax.plot(W[i], marker=mk, ms=3, lw=1, label=f'component {i+1}')
ax.set_xlabel('snapshot index k'); ax.set_ylabel(r'$w_k$ entry'); ax.set_title('First four
rows of W across snapshots')
ax.legend(); ax.grid(alpha=.3); plt.show()
```


First four rows of W across snapshots
50
0
−50
−100
−150
−200
−250
w_k entry
0
20
40
60
80
100
120
140
snapshot index k
component 1
component 2
component 3
component 4